

Design Doc: Escape Room Platform

Status: Current

Authors: Hephzibah Aggrey, Brianna Stewart, Danielle Lawrence, Gabrielle Williams

Background

The Escape Room Platform is a browser-based puzzle application that allows users to complete interactive escape room challenges through sequential or branching puzzle flows. Players solve puzzles, unlock hints, reveal answers after repeated failed attempts, and complete rooms under optional timed constraints.

The platform is intended to provide an engaging gameplay experience while demonstrating core software engineering principles such as modular architecture, database normalization, UML modeling, requirements traceability, and test-driven validation.

Overview

The system follows a layered architecture consisting of:

- **Presentation Layer** – HTML/CSS/JavaScript user interface
- **Application Layer** – Django views and request handling
- **Business Logic Layer** – `game_logic.py` services for puzzle flow and scoring
- **Persistence Layer** – Relational database models for users, puzzles, sessions, and analytics

The frontend provides a visually immersive room interface styled with a dark neon theme. The backend processes gameplay actions, validates user submissions, manages sessions, escalates hints, and records completion metrics.

This separation of concerns improves maintainability, scalability, and testability.

Detailed Design

1. Major Components

Frontend Module

The client-side interface renders the room environment and interactive objects using the provided CSS theme.

Key UI elements include:

- Dynamic room container
- Clickable puzzle objects
- Door unlock mechanism
- Answer submission form
- Real-time feedback messages
- Session progress indicators

The interface is responsive and designed for minimal latency between user input and backend response.

Backend Services

Core gameplay logic is implemented in `game_logic.py`.

Primary services include:

- `check_answer()` – Validates string, regex, and numeric responses
- `get_hint()` – Returns adaptive hints based on failure thresholds
- `update_progress()` – Updates attempts, hints used, and completed puzzles
- `get_next_puzzle()` – Resolves linear or branching puzzle transitions
- `check_completion()` – Determines challenge completion state
- `check_timeout()` – Enforces timed mode constraints
- `finalize_run()` – Persists completion statistics

This service-based design isolates business rules from HTTP request handling.

Persistence Layer / Database Design

Relational schema includes:

Entity	Description
User	Authenticated player accounts
EscapeRoom	Challenge metadata
Puzzle	Puzzle definitions and answer rules
Hint	Multi-level hints linked to puzzles
GameSession	Runtime progress per attempt
Leaderboard	Ranked completion times

Relationships

- One EscapeRoom has many Puzzles
- One Puzzle has many Hints
- One User has many GameSessions
- One GameSession belongs to one EscapeRoom

Database indexing should be applied to:

- user_id
- challenge_id
- completed
- total_time_seconds

for optimized leaderboard and analytics queries.

CRUD Use Cases

Resource	Create	Read	Update	Delete
User	Register account	View profile	Edit profile	Delete account
EscapeRoom	Create room	Browse rooms	Edit metadata	Remove room
Puzzle	Add puzzle	Retrieve puzzle	Modify answer logic	Delete puzzle
Hint	Add hint	Load hints	Edit hint text	Delete hint
Session	Start run	View progress	Update stats	Terminate run

UML Design Elements

State Diagram – Game Session Lifecycle

Idle → Initialized → Active → Puzzle Attempt

Transitions:

- Correct Answer → Next Puzzle
- Incorrect Answer → Hint Escalation
- Final Puzzle Solved → Completed
- Timer Expired → Timed Out

Sequence Diagram – Answer Submission Flow

1. User submits answer through UI
2. Frontend sends POST request to Django view
3. View invokes `check_answer()`
4. If invalid:
 - Increment attempts
 - Call `get_hint()` if threshold met
5. If valid:
 - Call `update_progress()`
 - Call `get_next_puzzle()`
6. Return JSON/UI response
7. Frontend re-renders puzzle state

Object Model

Puzzle Class

- `id`
- `title`
- `correct_answer`
- `match_type`
- `flow_type`
- `branches`
- `next_puzzle`

GameSession Class

- user
- challenge
- attempts
- hints_used
- revealed_answers
- completed
- timed_out
- start_time
- end_time

Design Logic / Formulas

Completion Percentage

$\text{completion_rate} = \text{completed_puzzles} / \text{total_puzzles} \times 100$

Hint Escalation Thresholds

- Level 1 Hint = 2 failed attempts
- Level 2 Hint = 4 failed attempts
- Level 3 Hint = 6 failed attempts

Testing Plan

Use Case	Test Type	Expected Result
Valid answer	Unit Test	Returns True
Invalid answer	Unit Test	Returns False
Hint unlock	Functional Test	Hint appears at threshold
Reveal answer	Functional Test	Unlocks after 8 failures
Branching puzzle	Integration Test	Correct route selected
Session completion	Integration Test	Session marked complete
Timeout	Functional Test	Session terminated

Security and Privacy

- Django authentication and hashed passwords
- CSRF protection on form submissions
- Input sanitization for regex/string handling
- Access control for user sessions
- Minimal personally identifiable data storage

Accessibility

- WCAG-aligned contrast ratios
- Keyboard-accessible controls
- Semantic labels for interactive elements
- Readable typography using Inter font
- Error/success feedback beyond color alone

Ethical Design Considerations

Fairness and Transparency

Puzzle evaluation logic uses deterministic rules applied equally across all users. Hint thresholds and timers are consistent and auditable.

User Privacy and Data Protection

Only necessary user data is stored. Sessions are isolated and protected using authenticated access patterns.

Environmental Sustainability

The platform uses lightweight frontend assets and efficient backend functions to minimize compute overhead and server resource consumption.

Future Goals

- Multiplayer cooperative sessions
- Procedural puzzle generation
- Real-time chat collaboration
- Mobile-native version
- Predictive analytics dashboard

Design Changes Over Time

Change	Justification
Added adaptive hint escalation	Reduced player abandonment
Added timed mode	Increased replayability
Added branching puzzle flow	Improved gameplay variation
Added reveal-answer fallback	Reduced frustration loops

Conclusion

The Escape Room Platform applies structured software design principles through modular services, normalized data models, UML artifacts, and traceable requirements mapping. The architecture supports current PRD objectives while remaining extensible for future features and increased scale.